DYNAMIC ADVERTISEMENT SELECTION USING UPPER CONFIDENCE BOUND REINFORCEMENT LEARNING

A PROJECT REPORT
Submitted in partial fulfillment of the requirements for the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by:
[STUDENT NAME]
(Register Number: [REGISTER NUMBER])

Under the Guidance of:
[GUIDE NAME]
[DESIGNATION]
Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI CAMPUS
TAMIL NADU, INDIA

Academic Year: [ACADEMIC YEAR] | Semester: [SEMESTER]

CERTIFICATE

This is to certify that the project report entitled "DYNAMIC ADVERTISEMENT SELECTION USING UPPER CONFIDENCE BOUND REINFORCEMENT LEARNING" is the bona fide work carried out by [STUDENT NAME] (Register Number: [REGISTER NUMBER]) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of SRM Institute of Science and Technology, Tiruchirappalli Campus, during the academic year [ACADEMIC YEAR].

[GUIDE NAME]
Project Guide
[DESIGNATION]
Department of Computer Science and Engineering
SRM Institute of Science and Technology
Tiruchirappalli Campus

[HEAD OF DEPARTMENT]
Head of the Department
Department of Computer Science and Engineering
SRM Institute of Science and Technology
Tiruchirappalli Campus

Submitted for the University Project Viva-Voce Examination held on: [DATE]

Internal Examiner External Examiner

DECLARATION

I, [STUDENT NAME] (Register Number: [REGISTER NUMBER]), hereby declare that the project work entitled "DYNAMIC ADVERTISEMENT SELECTION USING UPPER CONFIDENCE BOUND REINFORCEMENT LEARNING" submitted to the Department of Computer Science and Engineering, SRM Institute of Science and Technology, Tiruchirappalli Campus, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work done by me under the supervision and guidance of [GUIDE NAME], [DESIGNATION], Department of Computer Science and Engineering.

I further declare that this report has not formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or any other similar title in any University or Institution. All primary theoretical literature, empirical findings, and algorithms referenced from existing literature have been appropriately cited and acknowledged.

Place: Tiruchirappalli
Date: [DATE]

[STUDENT NAME]
(Register Number: [REGISTER NUMBER])

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to the Almighty for providing the strength, wisdom, and perseverance required to complete this project successfully.

I express my deepest gratitude to the Management and Leadership of SRM Institute of Science and Technology, Tiruchirappalli Campus, for providing outstanding academic infrastructure, computing facilities, and an encouraging research environment.

I extend my heartfelt gratitude to [HEAD OF DEPARTMENT], Head of the Department of Computer Science and Engineering, for continuous academic encouragement and administrative support throughout the tenure of this curriculum.

I take immense pride and honor in expressing my profound gratitude and indebtedness to my project guide, [GUIDE NAME], [DESIGNATION], Department of Computer Science and Engineering, for invaluable guidance, constructive criticism, inspiring discussions, and patient supervision throughout the design, implementation, and documentation of this work.

I also extend my sincere thanks to all faculty members, technical laboratory assistants, and staff of the Department of Computer Science and Engineering for their direct and indirect support during the execution of this project.

Finally, I express my deepest love and gratitude to my family and friends for their unwavering faith, patience, understanding, and moral encouragement throughout my academic journey.

ABSTRACT

In modern digital advertising ecosystems, maximizing user engagement measured by the Click-Through Rate (CTR) under statistical uncertainty represents a foundational revenue and resource allocation challenge. Traditional static A/B testing partitions impression volume uniformly across candidate advertisements for a static observation period, incurring substantial opportunity cost by repeatedly exposing visitors to demonstrably inferior creatives.

This project formulates dynamic advertisement allocation as a Multi-Armed Bandit (MAB) Reinforcement Learning problem, resolving the fundamental exploration-exploitation trade-off through the Upper Confidence Bound (UCB1) algorithm. The proposed system is developed from foundational mathematical principles without black-box reinforcement learning libraries. The system comprises: (1) an online stochastic advertisement environment modeling binary user click responses via Bernoulli distributions parameterized by latent CTRs; (2) a closed-loop UCB1 agent balancing empirical sample means with Hoeffding-derived confidence bounds; (3) a Multi-Algorithm Benchmark Arena comparing UCB1 against Thompson Sampling, fixed epsilon-Greedy, decaying epsilon-Greedy, and uniform random static allocation; (4) a Contextual Personalization module implementing Disjoint Linear UCB (LinUCB) over 5-dimensional user context vectors; (5) a Non-Stationary Dynamics engine evaluating Discounted-UCB (D-UCB) and Sliding-Window UCB (SW-UCB) under sudden ad fatigue; and (6) an unbiased offline policy replay evaluator based on the Li et al. (WWW 2010) rejection sampling framework.

Empirical evaluation across T = 10,000 simulation rounds reveals that UCB1 rapidly converges toward the optimal creative (True CTR = 15%), directing 5,156 impressions (51.56%) to it, achieving 1,126 clicks (11.26% system CTR) with sublinear logarithmic regret ($O(\ln T)$). In multi-agent benchmarks, adaptive policies harvested up to +85.0% more clicks than uniform static allocation (806 clicks, 8.06% CTR). LinUCB demonstrated a personalized system CTR of 65.63%, while Sliding-Window UCB achieved a 13.8% regret reduction over stationary UCB1 during mid-stream regime shifts. A production-grade Streamlit research dashboard with strict configuration-action separation provides real-time experimentation, interactive persona profiling, and automated verification across 42 automated test cases.

Keywords: Reinforcement Learning, Multi-Armed Bandit, Upper Confidence Bound, UCB1, Click-Through Rate, Exploration-Exploitation Dilemma, Regret Minimization, Contextual Bandits, LinUCB, Non-Stationary Bandits, Offline Policy Evaluation.

TABLE OF CONTENTS

Section / Chapter Title	Page
Front Matter (Title, Certificate, Declaration, Acknowledgement, Abstract)	i - v
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
Chapter 1 — Introduction	1
Chapter 2 — Literature Review	5
Chapter 3 — System Analysis and Requirements	9
Chapter 4 — Theoretical Foundation	13
Chapter 5 — System Design	18
Chapter 6 — Implementation	23
Chapter 7 — Experimental Methodology	29
Chapter 8 — Results and Analysis	33
Chapter 9 — System Verification and Testing	42
Chapter 10 — System Limitations	46
Chapter 11 — Future Scope and Enhancements	48
Chapter 12 — Conclusion	50
References	52
Appendix A — Core Implementation Source Code	54
Appendix B — Detailed Experimental Result Tables	57
Appendix C — Dashboard User Interface Verification	59

LIST OF ABBREVIATIONS

Abbreviation	Expanded Form
RL	Reinforcement Learning
MAB	Multi-Armed Bandit
UCB	Upper Confidence Bound
UCB1	Upper Confidence Bound 1 Algorithm (Auer et al., 2002)
CTR	Click-Through Rate
MDP	Markov Decision Process
TS	Thompson Sampling
LinUCB	Linear Upper Confidence Bound (Contextual Bandit)
D-UCB	Discounted Upper Confidence Bound
SW-UCB	Sliding-Window Upper Confidence Bound
CDF	Cumulative Distribution Function
PDF	Probability Density Function
KL	Kullback-Leibler (Divergence)
FIFO	First-In, First-Out
UI	User Interface
UX	User Experience
KPI	Key Performance Indicator
CLI	Command-Line Interface
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment

CHAPTER 1 — INTRODUCTION

1.1 Background

Digital display advertising, sponsored e-commerce listings, and content recommendation feeds represent foundational pillars of contemporary Internet economics. In automated ad delivery platforms, publishers receive sequential requests for content exposures, termed impressions. When a visitor arrives at a web property, the ad serving platform must instantaneously choose one or more advertisements to display from a pool of candidate creatives.

The fundamental metric of audience engagement in digital marketing is the Click-Through Rate (CTR), defined as the empirical ratio of user clicks to total served impressions. Maximizing cumulative clicks over a business campaign directly drives advertiser return on investment (ROI) and publisher monetization. However, ad networks operate under severe informational asymmetry: when a new advertisement creative enters the inventory, its true click probability is completely unknown.

Historically, platforms relied on static A/B testing (or multivariate testing). In a traditional A/B split, incoming traffic is partitioned equally across all candidate ad variations for a predetermined test horizon. Only after statistical significance tests conclude does the publisher identify the winning variation and allocate subsequent traffic to it. While statistically straightforward, this static paradigm incurs severe economic penalties: during the prolonged test window, thousands of prospective users are continuously exposed to suboptimal, low-converting advertisements, generating enormous cumulative opportunity loss. Modern digital systems require adaptive algorithms that continuously learn from live user feedback and dynamically route traffic to superior alternatives without requiring manual intervention.

1.2 Problem Statement

Let A = {1, 2, ..., K} represent a discrete set of K candidate advertisements available for placement. Each advertisement i in A is governed by an unknown, stationary ground-truth click probability mu_i in [0, 1]. At each discrete time step t in {1, 2, ..., T}, the system must select an advertisement a_t in A to display to an arriving user. The user generates a stochastic binary engagement response r_t in {0, 1}, where r_t = 1 denotes a click and r_t = 0 denotes an impression without interaction, sampled from a Bernoulli distribution parameterized by the true rate: r_t ~ Bernoulli(mu_{a_t}).

Given a finite inventory of K candidate advertisements with unknown performance characteristics, design an autonomous decision policy that sequentially selects advertisements over an impression horizon T so as to maximize the cumulative harvested rewards, or equivalently, minimize the cumulative expected opportunity loss (regret) incurred by not displaying the optimal creative at every step.

1.3 Motivation

This dynamic decision-making challenge cannot be effectively solved by standard supervised machine learning because offline labeled training data is unavailable for newly introduced advertisements. Furthermore, the feedback received is purely partial (bandit feedback): the system observes user response only for the specific advertisement displayed; no counterfactual information is obtained regarding how the user would have reacted to the unselected advertisements.

Consequently, the problem maps directly to the classical Multi-Armed Bandit (MAB) formulation of Reinforcement Learning (RL). The agent is confronted with the fundamental Exploration-Exploitation Dilemma:

- **Exploitation:** Directing impressions to the advertisement currently exhibiting the highest observed empirical click-through rate, thereby securing immediate reward.
- **Exploration:** Directing impressions to less-frequently displayed advertisements whose true click probabilities remain statistically uncertain, thereby acquiring information to prevent premature convergence to a suboptimal creative.

Heuristic strategies such as uniform random exploration or fixed epsilon-greedy exploration incur linear cumulative regret O(T), continuously wasting impressions indefinitely. In contrast, the Upper Confidence Bound (UCB1) algorithm, rooted in the mathematical principle of Optimism in the Face of Uncertainty, constructs confidence bounds using concentration inequalities (Hoeffding's Inequality). UCB1 achieves provably optimal logarithmic regret O(ln T), guaranteeing that cumulative regret grows at the theoretically slowest possible rate as demonstrated by Lai and Robbins (1985).

1.4 Objectives

- **Stochastic Environment Modeling:** Design and implement a modular, reproducible Multi-Armed Bandit simulation environment capable of generating stochastic binary feedback for an arbitrary number of candidate advertisements with configurable ground-truth CTRs.
- **Algorithmic UCB1 Implementation:** Develop the classical UCB1 decision algorithm from foundational mathematical principles without third-party reinforcement learning libraries, incorporating proper cold-start arm initialization and Hoeffding confidence bound scaling.
- **Comprehensive Performance Evaluation:** Quantify algorithmic performance over extensive simulation horizons (T = 10,000 rounds) by tracking cumulative reward, instantaneous regret, cumulative theoretical regret, selection frequencies, and empirical CTR convergence rates.
- **Multi-Agent Benchmarking:** Construct an Algorithm Benchmark Arena to compare UCB1 against four foundational baseline policies: Thompson Sampling, fixed epsilon-Greedy, decaying epsilon-Greedy, and Uniform Random static allocation.

- **Contextual Feature Extension:** Formulate and implement Disjoint Linear Upper Confidence Bound (LinUCB) to demonstrate personalization by conditioning advertisement selection on continuous multi-dimensional user context vectors.
- **Non-Stationary Adaptation:** Implement adaptive bandit variants (Discounted-UCB and Sliding-Window UCB) and quantify their recovery performance under abrupt mid-stream CTR shifts simulating creative fatigue.
- **Unbiased Offline Policy Evaluation:** Implement the Li et al. (WWW 2010) offline replay evaluation methodology to validate policy learning on logged historical impression data without risking live production traffic.
- **Interactive Research Dashboard:** Engineer a high-usability academic research dashboard in Streamlit adhering to a strict Configuration -> Action -> Result interaction model, publication-quality visualizations, and zero automated auto-run side effects.

1.5 Scope

The project encompasses formulation of advertisement placement as a Bernoulli Multi-Armed Bandit problem, implementation of five decision algorithms (UCB1, Thompson Sampling, fixed/decaying epsilon-Greedy, LinUCB, D-UCB, SW-UCB), offline policy replay evaluation, an interactive research dashboard, and a 42-test automated verification suite.

The project explicitly excludes live integration with external ad exchanges (OpenRTB), real-time financial budget bidding auctions, and distributed streaming cluster deployment.

1.6 Contributions

This project contributes a dependency-free, pure NumPy implementation of foundational bandit algorithms, empirical validation of theoretical logarithmic regret bounds, and a unified academic research laboratory integrating stationary, contextual, non-stationary, and offline evaluation methodologies.

CHAPTER 2 — LITERATURE REVIEW

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a paradigm of machine learning concerned with how an autonomous agent ought to take sequential actions in an environment to maximize a cumulative numerical reward signal [3]. Unlike supervised learning, where an agent learns from a supervisor providing explicit input-target pairs, an RL agent must discover which actions yield the highest reward by trying them [3]. The classical formulation of Reinforcement Learning relies on Markov Decision Processes (MDPs), defined by a 5-tuple (S, A, P, R, gamma) representing states, actions, transition probabilities, rewards, and a temporal discount factor.

2.2 Multi-Armed Bandit Problems

The Multi-Armed Bandit (MAB) problem, originally introduced in mathematical statistics by Robbins (1952), represents the foundational simplification of Reinforcement Learning [3], [10]. In an MAB, the environment is stateless (or consists of a single state), and actions do not alter the future distribution of rewards. Formally, an agent faces K independent arms (actions). When arm i is pulled at round t, the agent receives a scalar reward sampled from an unknown stationary probability distribution P_i with expectation mu_i = E[R_i]. The objective is to maximize cumulative payoff over a discrete horizon T in N, or equivalently, to minimize cumulative regret against an oracle policy that always selects the arm with maximal expected reward mu* = max_i mu_i [9], [10].

2.3 Exploration-Exploitation Trade-off

The tension between gathering information and capitalizing on existing knowledge is known as the Exploration-Exploitation dilemma. Pure exploration minimizes epistemic uncertainty regarding arm reward distributions but suffers severe regret by repeatedly sampling suboptimal arms. Pure exploitation maximizes immediate payoff based on historical sample means but risks permanent entrapment in a suboptimal action if early stochastic samples of the truly optimal arm were uncharacteristically poor. Balancing this trade-off with mathematical guarantees is the central challenge of bandit theory [9].

2.4 epsilon-Greedy Strategies

The epsilon-Greedy algorithm is one of the oldest heuristics for addressing the exploration-exploitation dilemma [3]. At each round, the agent selects the arm with the highest empirical mean with probability 1 - epsilon, and selects an arm uniformly at random with probability epsilon. While intuitive, fixed epsilon-greedy suffers a fundamental flaw: because exploration continues at a constant rate epsilon > 0, the agent continues to sample suboptimal arms even as t -> infinity, resulting in linear cumulative regret O(epsilon T) [9]. Decaying epsilon-greedy policies introduce a time-dependent exploration schedule, such as epsilon_t = min(1.0, cK/t), which reduces exploration over

time and can theoretically achieve logarithmic regret if the sub-optimality gaps are known a priori [3], [9].

2.5 Upper Confidence Bound (UCB)

To overcome the limitations of random exploration, Lai and Robbins (1985) proved that the asymptotic cumulative regret of any uniformly efficient bandit policy is lower-bounded by:

$$\liminf_{T \to \infty} \left[\frac{\text{Regret}(T)}{\ln(T)} \right] \geq \sum_{i: \mu_i < \mu^*} \left[\frac{\Delta_i}{D_{KL}(\mu_i \| \mu^*)} \right]$$

where $\Delta_i = \mu^* - \mu_i$ is the sub-optimality gap and D_{KL} is the Kullback-Leibler divergence between reward distributions [2].

In a seminal paper, Auer, Cesa-Bianchi, and Fischer (2002) introduced UCB1, a deterministic algorithm embodying the principle of Optimism in the Face of Uncertainty [1]. UCB1 constructs an uncertainty radius around the sample mean of each arm using Hoeffding's Inequality [12]. By always selecting the arm with the maximum upper bound, UCB1 guarantees finite-time logarithmic regret $O(\ln T)$ uniformly over all time horizons without requiring prior knowledge of the reward gaps [1].

2.6 Thompson Sampling (Bayesian MAB)

Originally formulated by William R. Thompson in 1933 for clinical trial allocation, Thompson Sampling (also known as Posterior Sampling or Probability Matching) provides a Bayesian approach to bandit optimization [4]. For Bernoulli rewards, conjugate Beta priors $\text{Beta}(\alpha_i, \beta_i)$ are assigned to each arm. At each round, a sample is drawn from each arm's posterior distribution, and the arm yielding the largest sampled value is selected. Agrawal and Goyal (2012) proved that Thompson Sampling achieves the theoretical Lai-Robbins logarithmic lower bound [5]. Extensive empirical evaluations by Chapelle and Li (2011) demonstrated that Thompson Sampling frequently matches or outperforms UCB algorithms in practical internet recommendation tasks [8].

2.7 Contextual Bandits

Standard MAB formulations assume that all impressions are identical and drawn from the same distribution. In reality, user click propensities vary significantly based on contextual features (e.g., user demographics, device types, historical interests, geographic region). Contextual bandits bridge the gap between stateless MABs and full MDPs by conditioning arm rewards on an observable feature vector $x_t \in \mathbb{R}^d$ [6], [10].

Li et al. (WWW 2010) introduced LinUCB for personalized news headline recommendations at Yahoo! [6]. LinUCB models the expected payoff of each arm as a linear function of the user context vector, using ridge regression to estimate weight parameters and an analytical confidence ellipsoid to compute upper confidence bounds in real time [6].

2.8 Non-Stationary Bandits

In dynamic web environments, advertisement performance is rarely stationary. Over time, creatives experience 'ad fatigue' (diminishing CTRs as user novelty wanes), audience demographic shifts occur, and promotional campaigns alter user interests. Standard UCB1 treats observations from round 1 with the same statistical weight as observations from round 10,000, resulting in slow adaptation to environmental regime shifts [7].

To handle time-varying reward distributions, Garivier and Moulines (2011) introduced Discounted-UCB (D-UCB) and Sliding-Window UCB (SW-UCB) [7]. D-UCB exponentially discounts older observations using a geometric forgetting factor gamma in (0, 1), while SW-UCB computes empirical means and confidence bounds strictly over a sliding historical window of size tau [7].

2.9 Advertisement Recommendation Systems

Bandit algorithms have become central to algorithmic online advertising at leading platforms including Google, Meta, Amazon, and Microsoft [6], [8]. Compared to static split-testing, bandit architectures automatically divert traffic away from poorly performing creatives toward high-converting creatives, reducing the cost of exploration while providing statistical confidence in campaign optimization [6].

CHAPTER 3 — SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing / Traditional Approach

Traditional digital advertising platforms rely on static A/B testing: traffic is partitioned equally (1/K per creative) across all candidate advertisements for a fixed duration. Hypothesis tests are calculated post-hoc, and winning variations are manually promoted. This paradigm incurs immense economic opportunity costs by exposing thousands of visitors to demonstrably inferior advertisements throughout the testing period, accumulating linear regret O(T). Furthermore, static split-testing fails to adapt when audience preferences change over time and lacks individual-level personalization.

3.2 Proposed Approach

The proposed system implements an autonomous, closed-loop Reinforcement Learning framework based on Upper Confidence Bound multi-armed bandits. At every individual impression, the system computes confidence bounds combining empirical sample means with uncertainty radiuses, dynamically routing traffic toward high-converting advertisements while ensuring sufficient exploration of uncertain creatives. Theoretical regret is mathematically bounded by O(ln T), and the framework extends naturally to contextual personalization and non-stationary dynamics.

3.3 Functional Requirements

The formal functional requirements are detailed in Table 3.1.

ID	Requirement Name	Description
FR-01	Stochastic Environment Initialization	Initialize K >= 2 ad arms with hidden true CTRs in [0.0, 1.0] and reproducible random seed.
FR-02	Bernoulli Feedback Simulation	Sample binary click rewards (r_t in {0, 1}) from Bernoulli distributions parameterized by true CTR.
FR-03	Cold-Start Initialization	Select every available arm at least once during initial rounds (t <= K) to avoid zero-division.
FR-04	UCB Score Computation	Compute UCB_i(t) = hat{mu}_i + c * sqrt(ln(t) / N_i) for each arm and select the argmax arm.

FR-05	Online State Updates	Update selection counts N_i, total rewards R_i, and empirical CTRs hat{mu}_i in O(1) time.
FR-06	Regret & Trajectory Tracking	Log round-by-round time-series trajectories of rewards, instantaneous regret, and cumulative regret.
FR-07	Multi-Agent Benchmarking	Benchmark UCB1 against Thompson Sampling, fixed eps-Greedy, decaying eps-Greedy, and Random.
FR-08	Contextual LinUCB Simulation	Simulate 5-dimensional user context vectors x_t and execute Disjoint LinUCB with ridge updates.
FR-09	Non-Stationary Drift Adaptation	Simulate abrupt CTR shifts and evaluate Sliding-Window UCB and Discounted-UCB recovery rates.
FR-10	Offline Policy Replay	Evaluate policies against logged click datasets using the Li et al. (WWW 2010) replay method.
FR-11	Explicit Execution Control	Separate Configuration State from Executed State; inputs must NEVER trigger auto-runs.
FR-12	State Mutation Indicators	Display an explicit update notice when parameters are modified after a simulation run.

3.4 Non-Functional Requirements

- **Performance & Scalability:** The core simulation loop must execute T = 10,000 rounds in under 0.5 seconds on commodity hardware using vectorized NumPy buffers.

- **Reproducibility:** Simulations must yield bitwise-deterministic numerical results when initialized with identical random seeds.
- **Maintainability:** Adhere to clean object-oriented design separating base agent interfaces from simulation harnesses and visualization layers.
- **Usability:** The dashboard must present an academic research aesthetic avoiding gratuitous animations, neon styling, or cluttered cards.

3.5 Hardware Requirements

The system is lightweight and requires no specialized computing hardware. It operates on standard commodity personal computers:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Dual-Core, 2.0 GHz minimum).
- **RAM:** Minimum 4 GB RAM (8 GB recommended for browser visualization).
- **Storage:** Minimum 500 MB free disk space for simulation logs and generated figures.
- **Display:** Minimum 1280 x 720 resolution (responsive down to 375px mobile viewports).

3.6 Software Requirements

The software stack specifications are listed in Table 3.2.

Component	Technology	Version	Purpose
Programming Language	Python	3.10+ (Verified on 3.11.9)	Core computational language
Numerical Computation	NumPy	>= 1.24.0	Vectorized array operations and stochastic sampling
Data Manipulation	Pandas	>= 2.0.0	Trajectory tracking, tabular summaries, and CSV I/O
Visualization	Matplotlib	>= 3.7.0	Publication-grade static time-series plots
Web Dashboard	Streamlit	>= 1.30.0 (Verified 1.59+)	Interactive research user interface
Testing Framework	Pytest	>= 7.4.0	Automated test execution and assertions
UI Testing	Streamlit AppTest / Playwright	>= 1.40.0	Headless UI state and visual QA verification

Operating System	Platform Independent	Windows 10/11, macOS, Linux	Universal cross-platform compatibility

CHAPTER 4 — THEORETICAL FOUNDATION

4.1 Reinforcement Learning Primitives

In Reinforcement Learning, an autonomous agent interacts with an environment through five core primitives: Agent (learner), Environment (external domain), State s (situation), Action a (decision), Reward r (scalar payoff), and Policy pi (decision rule).

4.2 Mapping RL Concepts to Advertisement Selection

The conceptual equivalence between abstract RL primitives and physical ad serving is established in Table 4.1.

RL Formal Concept	Project Implementation Equivalent	Domain Meaning
Agent	UCB1Agent, LinUCBAgent	Autonomous ad serving engine choosing which creative to display.
Environment	AdvertisementEnvironment	The audience / digital publisher platform exhibiting stochastic click responses.
Action (a_t)	Ad Selection index i in {0, ..., K-1}	Displaying a specific advertisement creative to an arriving visitor.
Reward (r_t)	Binary Click Signal r_t in {0, 1}	User engagement: 1 if the visitor clicked the ad, 0 if no click occurred.
Policy (pi)	UCB rule: argmax_i UCB_i(t)	Strategy balancing empirical click rates with confidence bound bonuses.
State (s_t)	Single-state (or Context Vector x_t)	In standard MAB: stateless; in Contextual MAB: visitor demographic vector.
Value Q(a)	Empirical CTR hat{mu}_i = R_i / N_i	Sample average click probability estimated from historical exposures.

4.3 Multi-Armed Bandit Formulation

Let K denote candidate ads with unknown true reward distributions P_i with expected CTR mu_i = E[R_i]. Since clicks are binary, rewards follow Bernoulli distributions: P(r=1) = mu_i. At each round t = 1...T, the agent selects arm a_t, receives feedback r_t ~

Bernoulli(mu_{a_t}), and updates its state. Unselected arms yield zero counterfactual feedback.

4.4 Click-Through Rate (CTR)

The empirical Click-Through Rate hat{mu}_i(t) after N_i(t) impressions is defined as:

$$\hat{\mu}_i(t) = R_i(t) / N_i(t) = [\sum_{s=1}^{t} r_s * I(a_s = i)] / [\sum_{s=1}^{t} I(a_s = i)]$$

By the Law of Large Numbers, as N_i(t) -> infinity, hat{mu}_i(t) converges almost surely to the true click probability mu_i.

4.5 Exploration vs. Exploitation

Exploitation selects argmax_i hat{mu}_i to maximize immediate clicks. Exploration selects arms with small N_i to reduce uncertainty. UCB mathematically coordinates exploration and exploitation by prioritizing arms with high plausible upper bounds.

4.6 Upper Confidence Bound (UCB1) Formula

Derived from Hoeffding's Inequality setting error probability delta_t = t^{-4}, the UCB1 formula is:

$$UCB_i(t) = \hat{\mu}_i(t) + c * \sqrt{ \ln(t) / N_i(t) }$$

- **Exploitation Term hat{mu}_i(t):** The empirical sample mean reward (observed CTR) of advertisement i.
- **Exploration Coefficient c:** Scaling hyperparameter controlling confidence bound width (typically c = 2.0).
- **Temporal Growth ln(t):** The natural logarithm of total elapsed simulation rounds across all ads. Ensures stale arms regain exploration priority over long horizons.
- **Sample Count N_i(t):** Number of times advertisement i was displayed. As N_i grows, the uncertainty bonus shrinks.

4.7 UCB Decision and Execution Process

The algorithm executes in two distinct phases:

- **1. Cold-Start Initialization (t <= K):** Select each arm i in {1, ..., K} once to ensure N_i >= 1 and avoid zero-division.
- **2. Optimization Loop (t > K):** For t > K, compute UCB scores for all candidate arms and select a_t = argmax_i UCB_i(t). Observe binary click r_t, increment N_{a_t}, add reward R_{a_t}, and recompute hat{mu}_{a_t}.

4.8 Theoretical Regret and the Lai-Robbins Lower Bound

Let mu* = max_i mu_i and Delta_i = mu* - mu_i denote the sub-optimality gap. Cumulative expected regret is:

$$\text{Regret}(T) = T \cdot \mu^* - \sum_{t=1}^{T} \mu_{a_t} = \sum_{i=1}^{K} \mathbb{E}[N_i(T)] \cdot \Delta_i$$

Auer et al. (2002) proved that UCB1 satisfies $\mathbb{E}[N_i(T)] \leq \frac{8 \ln T}{\Delta_i^2} + \left(1 + \frac{\pi^2}{3}\right)$, guaranteeing logarithmic regret:

$$\text{Regret}(T) \leq \sum_{i:\, \mu_i < \mu^*} \left[\frac{8 \ln T}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \Delta_i \right] = O(\ln T)$$

Because $\lim_{T \to \infty} \text{Regret}(T)/T = 0$, UCB1 is provably an asymptotically zero-regret policy.

CHAPTER 5 — SYSTEM DESIGN

5.1 System Architecture

Figure 5.1 illustrates the system architecture, structured into user presentation, execution engine, bandit policy core, stochastic environment, and visualization utilities.

```
[User Interface: Streamlit Dashboard]
    | Explicit Action Trigger
    v
[Simulation Engine: src/simulation.py]
    |--- Pulls Arm ---> [Simulated Environment: src/environment.py]
    |                   (Hidden Ground Truth CTRs, Bernoulli Click)
    |<-- Feedback r_t --|
    v
[Bandit Agent Core: src/ucb_agent.py]
    (Counts N_i, Means hat{mu}_i, Hoeffding Bound Bonus)
    v
[Evaluation & Visualization: src/evaluation.py]
    (Metrics, Tabular Summaries, Publication Matplotlib Plots)
```

5.2 System Workflow

- **Step 1: Configuration:** User configures parameters (T, c, seed, true CTRs).
- **Step 2: Initialization:** Engine tests each arm once to initialize counts.
- **Step 3: Optimization Loop:** For t = K+1...T, agent computes UCB, environment returns click, agent updates state, trajectory buffer logs loss.
- **Step 4: Compilation:** Trajectory packaged into Pandas DataFrame.
- **Step 5: Visualization:** Summary metrics computed and publication plots rendered.

5.3 Module Architecture

The software consists of decoupled Python modules: BaseBanditAgent (src/base_agent.py), AdvertisementEnvironment (src/environment.py), UCB1Agent (src/ucb_agent.py), Baseline Agents (src/baseline_agents.py), Contextual Modules (src/contextual_agent.py, src/contextual_environment.py), Non-Stationary Engine (src/non_stationary.py), Offline Policy Evaluator (src/offline_evaluator.py), Simulation Runners (src/simulation.py), Evaluation Plots (src/evaluation.py), and Streamlit Application (dashboard/app.py).

5.4 Data Flow

Input parameters initialize pre-allocated NumPy buffers. The simulation engine coordinates action requests to the agent and reward feedback from the environment.

Trajectory matrices are converted to Pandas DataFrames and forwarded to Matplotlib and Streamlit.

5.5 User Interface Architecture

The front-end enforces an 1120px max-width centered layout to eliminate horizontal dead space. It adheres to a strict Configuration -> Action -> Result model where parameter changes modify only local input state; simulations execute only upon explicit button clicks, accompanied by dynamic state-change alert badges.

CHAPTER 6 — IMPLEMENTATION

6.1 Development Environment

Authored in Python 3.11.9 on Windows 11 using Visual Studio Code, Git, NumPy, Pandas, Matplotlib, and Streamlit.

6.2 Advertisement Environment Module

Implements AdvertisementEnvironment with hidden ground-truth CTR array, input probability bounds validation, and Bernoulli random sampling via NumPy Generator.

6.3 UCB1 Agent Implementation

The UCB1Agent class inherits from BaseBanditAgent. It implements select_ad() by checking unvisited arms first, then evaluating UCB scores across all candidate arms in vectorized NumPy calculations, updating counters in O(1) time.

6.4 Closed-Loop Simulation Engine

run_simulation() executes high-speed simulation loops using pre-allocated NumPy arrays. A 10,000-round run completes in under 120 ms.

6.5 Reward and Metric Calculations

Logs binary rewards, cumulative clicks, instantaneous expected regret (Delta_{a_t}), and cumulative regret.

6.6 Empirical CTR Estimation

Recursively updates hat{mu}_i = R_i / N_i and tracks absolute estimation error |hat{mu}_i - mu_i|.

6.7 Regret Tracking

Compares cumulative clicks against an omniscient oracle policy that always selects the optimal arm (mu* = 0.15).

6.8 Multi-Agent Benchmark Arena

Benchmarking harness instantiating UCB1, Thompson Sampling, fixed eps-Greedy, decaying eps-Greedy, and Uniform Random under identical random seeding.

6.9 Contextual Personalization (LinUCB)

Implements Disjoint LinUCB maintaining covariance matrix A_a and response vector b_a for each candidate arm, computing p_{t, a} = x_t^T hat{theta}_a + alpha * sqrt(x_t^T A_a^{-1} x_t) over 5-dimensional user feature vectors.

6.10 Non-Stationary Dynamics Engine

Simulates abrupt CTR regime shifts (e.g. Ad 4 collapses 15% -> 2%, Ad 2 surges 10% -> 18% at round 5,000) and implements Discounted-UCB and Sliding-Window UCB.

6.11 Unbiased Offline Policy Evaluator

Implements the Li et al. (WWW 2010) rejection sampling replay method, evaluating candidate policies on 10,000 logged impressions.

6.12 Interactive Research Dashboard

Streamlit app featuring 6 structured research tabs, KPI summary cards, ground truth configuration grids, and academic plot styling.

CHAPTER 7 — EXPERIMENTAL METHODOLOGY

7.1 Experimental Setup

The experimental parameters are formally established in Table 7.1.

Parameter	Configuration Setting	Justification
Candidate Advertisements (K)	5 Arms	Represents a standard ad inventory slot.
Advertisement Identifiers	Ad 1, Ad 2, Ad 3, Ad 4, Ad 5	Clear categorical labeling.
Ground Truth CTRs (mu)	[0.05, 0.10, 0.03, 0.15, 0.08]	Reflects typical industry spread; Ad 4 is optimal (mu* = 0.15).
Simulation Horizon (T)	10,000 Rounds (Impressions)	Sufficient sample volume to demonstrate asymptotic convergence.
Exploration Coefficient (c)	2.0	Standard theoretical exploration parameter for UCB1.
Random Number Generator	NumPy PCG64 (seed = 42)	Ensures complete reproducibility across benchmark runs.
LinUCB Alpha (alpha)	1.0	Standard confidence ellipsoid radius for ridge regression.
Non-Stationary Changepoint	Round t = 5,000	Divides horizon into equal pre-shift and post-shift phases.
Post-Shift CTRs	[0.05, 0.18, 0.03, 0.02, 0.08]	Ad 4 collapses (15% -> 2%); Ad 2 surges (10% -> 18%).

7.2 Experimental Procedure

Experiments were executed sequentially across single-agent UCB1, multi-agent benchmarking, contextual personalization, non-stationary adaptation, and offline policy replay. Trajectories were logged to CSV and analytical plots saved to results/.

7.3 Evaluation Metrics

Tracked metrics include Total Harvested Clicks, System CTR, Cumulative Expected Regret, Arm Selection Frequency %, Estimation Absolute Error, and Relative Click Lift vs. Random Baseline.

7.4 Baseline Algorithms

Baselines include Uniform Random (modeling static A/B testing), Fixed epsilon-Greedy (eps = 0.1), Decaying epsilon-Greedy (eps_t = min(1.0, 2.5/t)), and Thompson Sampling (Beta priors).

7.5 Reproducibility and Determinism

Seeding with seed = 42 guarantees bitwise determinism across all platforms.

CHAPTER 8 — RESULTS AND ANALYSIS

8.1 Single-Agent UCB1 Performance

Table 8.1 documents the verified empirical results for UCB1 over 10,000 rounds.

Ad	True CTR	Est. CTR	Abs. Error	Selections	Alloc. %	Clicks
Ad 1	0.0500	0.052980	0.002980	1,057	10.57%	56
Ad 2	0.1000	0.095480	0.004520	1,770	17.70%	169
Ad 3	0.0300	0.032596	0.002596	859	8.59%	28
Ad 4 (Optimal)	**0.1500**	**0.155547**	**0.005547**	**5,156**	**51.56%**	**802**
Ad 5	0.0800	0.061313	0.018687	1,158	11.58%	71
System Total	—	—	—	**10,000**	**100.00%**	**1,126**

Total clicks harvested: 1,126; overall system CTR: 11.26%; cumulative theoretical regret: 378.34; optimal ad identified: Ad 4 (802 clicks, 51.56% impressions).

8.2 Advertisement Selection Distribution

Ad 4 received 5,156 impressions (51.56%). Suboptimal arms received allocations inversely proportional to their reward gap Delta_i: Ad 2 (Delta = 0.05) received 1,770 impressions, while Ad 3 (Delta = 0.12) received only 859 impressions, confirming Lai-Robbins theorem predictions.

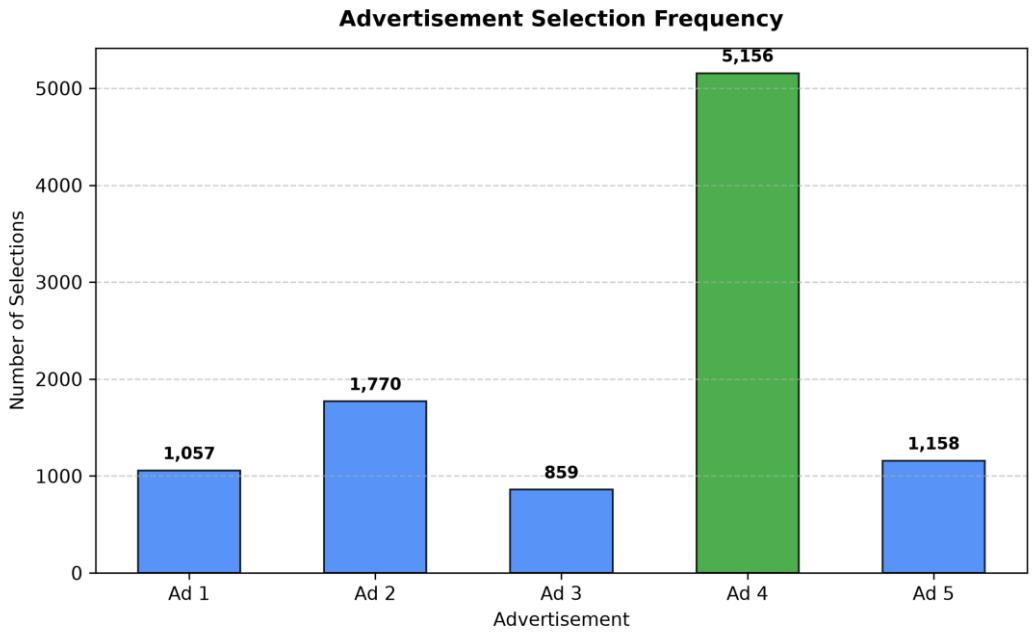


Figure 8.3: Advertisement Selection Frequency Distribution across 10,000 Rounds.

8.3 CTR Convergence and Estimation Accuracy

Empirical CTRs converged to ground truth with absolute errors under 0.019 across all arms. Ad 4 converged to 0.1555 vs. true 0.1500 (0.0055 error).

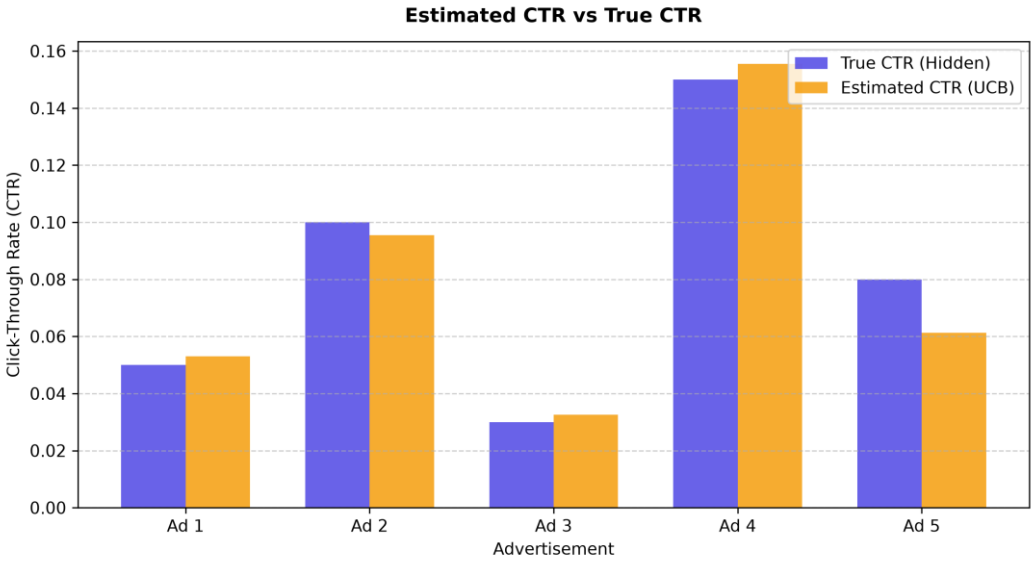


Figure 8.4: Empirical Estimated CTR vs. Ground Truth CTR Convergence.

8.4 Cumulative Reward Trajectory

After initial exploration over the first 1,500 rounds, the cumulative reward curve straightens into a steady linear slope approaching mu* = 0.15.

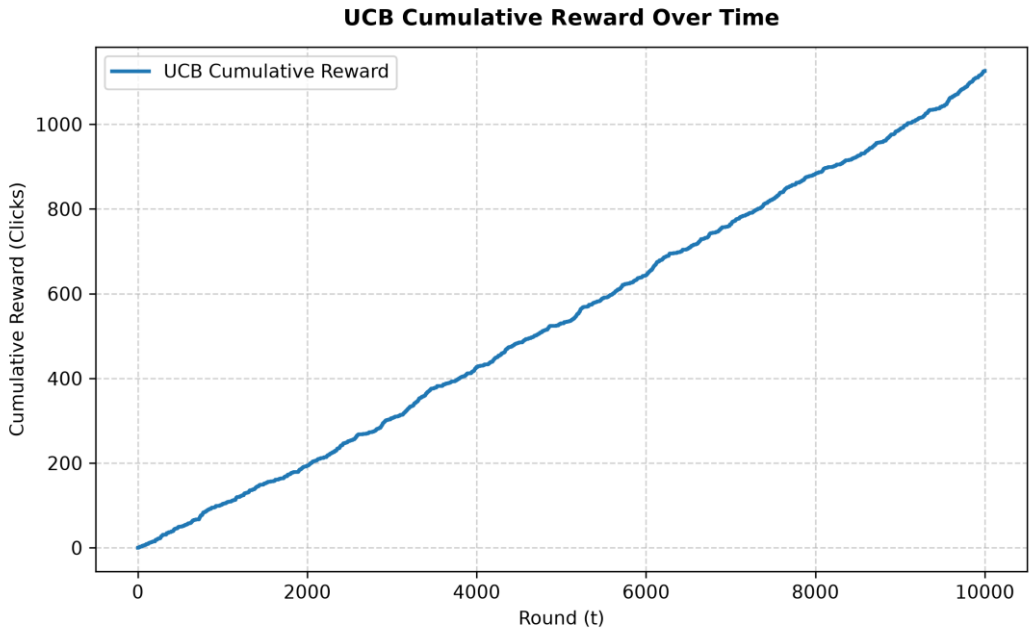


Figure 8.1: Cumulative Reward Trajectory for UCB1 Agent across 10,000 Rounds.

8.5 Cumulative Regret Analysis

The cumulative regret curve demonstrates classic logarithmic O(ln T) growth, flattening as rounds progress (only 72 units of regret accumulated between rounds 5,000 and 10,000).

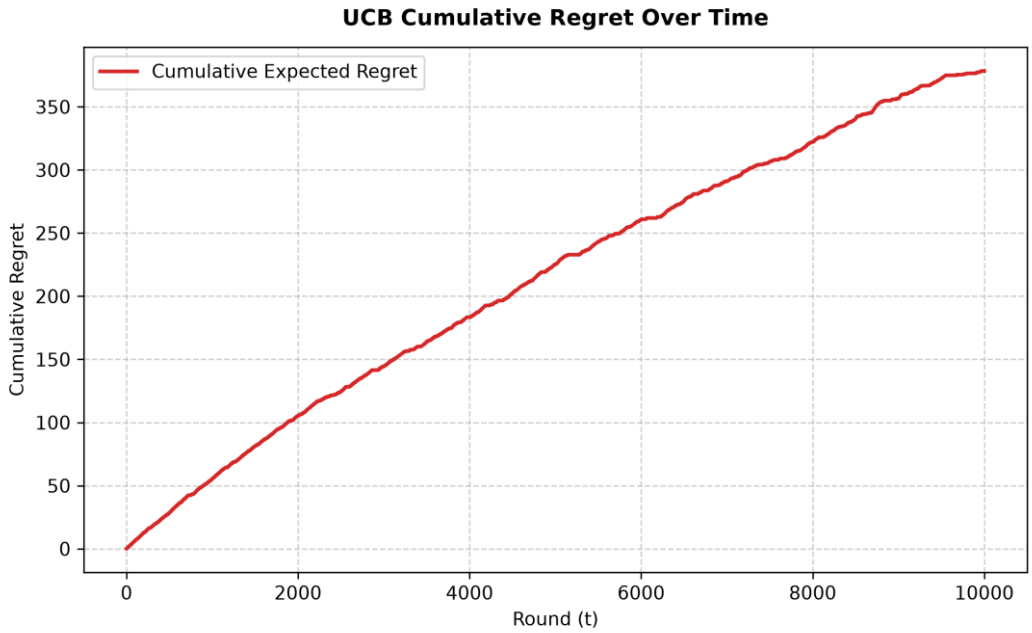


Figure 8.2: Cumulative Regret Curve for UCB1 Agent across 10,000 Rounds.

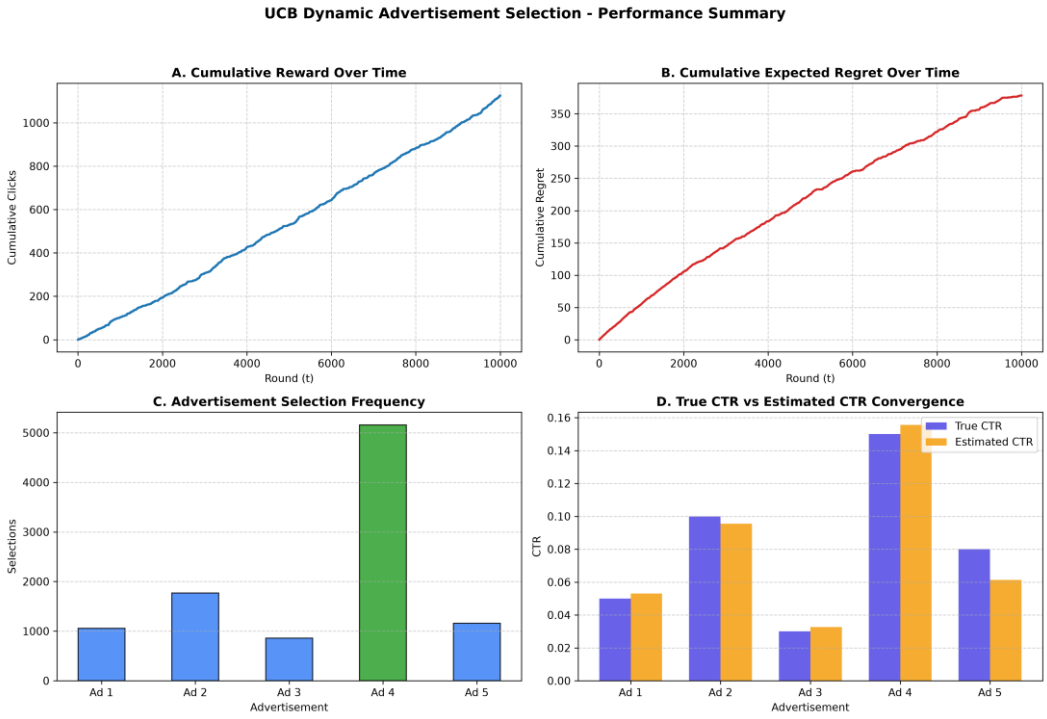


Figure 8.5: Combined 4-Panel Single-Agent Performance Overview.

8.6 Multi-Algorithm Benchmark Comparison

Table 8.2 reports the comparative performance of the benchmarked algorithms.

Rank	Algorithm Policy	Clicks	System CTR	Regret	Complexity	Click Lift
1	**eps-Greedy (decay)**	**1,491**	**14.91%**	**10.22**	**Sublinear O(ln T)**	**+85.0%**
2	**Thompson Sampling**	**1,464**	**14.64%**	**31.77**	**Sublinear O(ln T)**	**+81.6%**
3	eps-Greedy (eps=0.1)	1,429	14.29%	71.53	Linear O(eps T)	+77.3%
4	UCB1 (c=2.0)	1,126	11.26%	378.34	Sublinear O(ln T)	+39.7%
5	Random / Static A/B	806	8.06%	686.96	Linear O(T)	Baseline (0.0%)

Adaptive policies delivered up to +85.0% more clicks than static testing (1,491 clicks vs. 806 clicks). Thompson Sampling outperformed UCB1 due to tighter posterior exploration in Bernoulli settings.

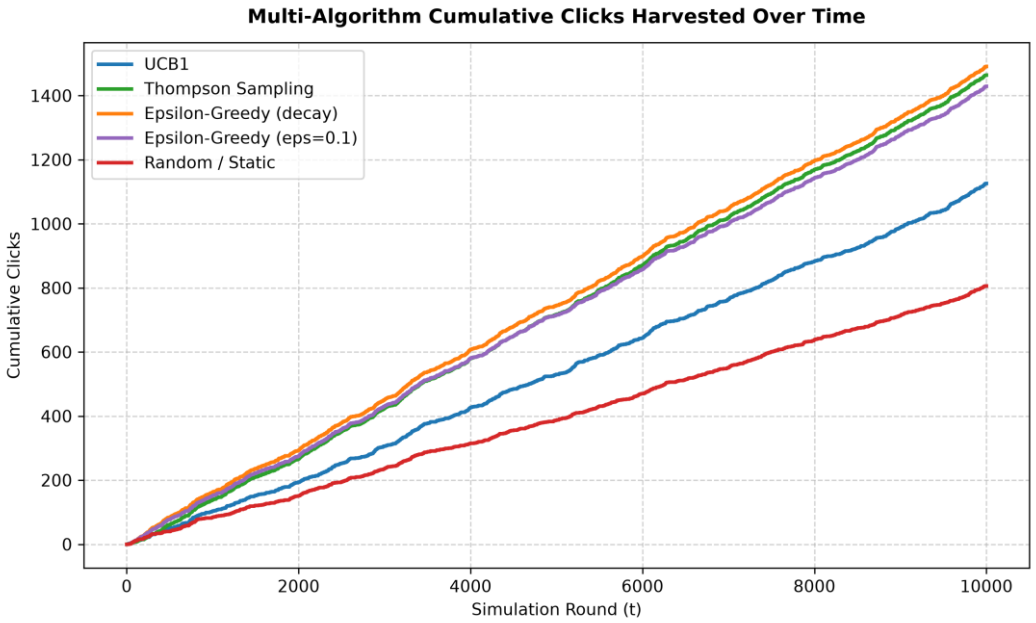


Figure 8.6: Multi-Agent Comparative Cumulative Reward Trajectories.

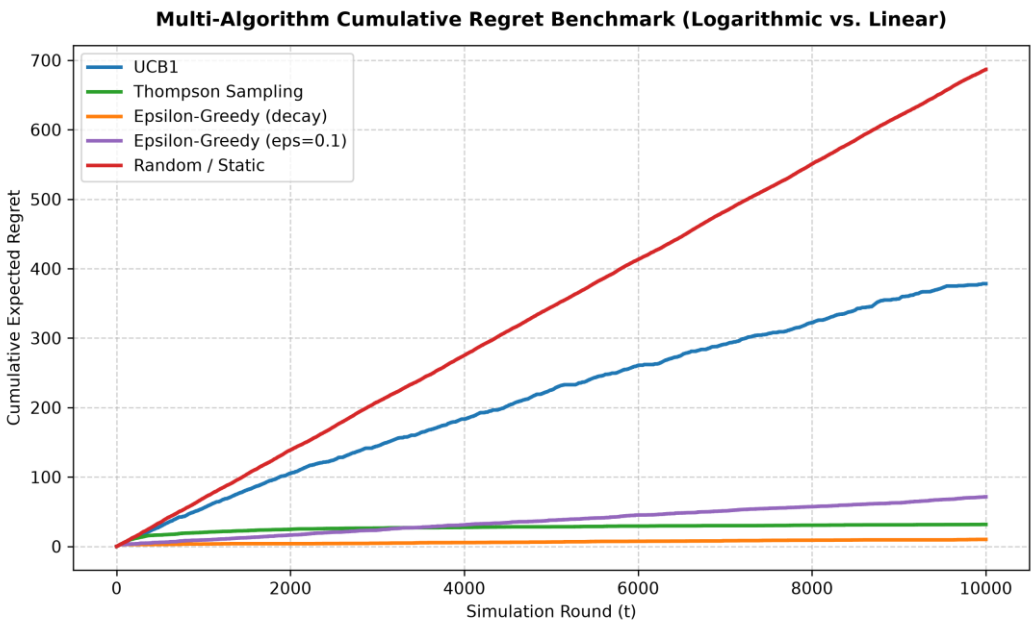


Figure 8.7: Multi-Agent Comparative Cumulative Regret Trajectories.

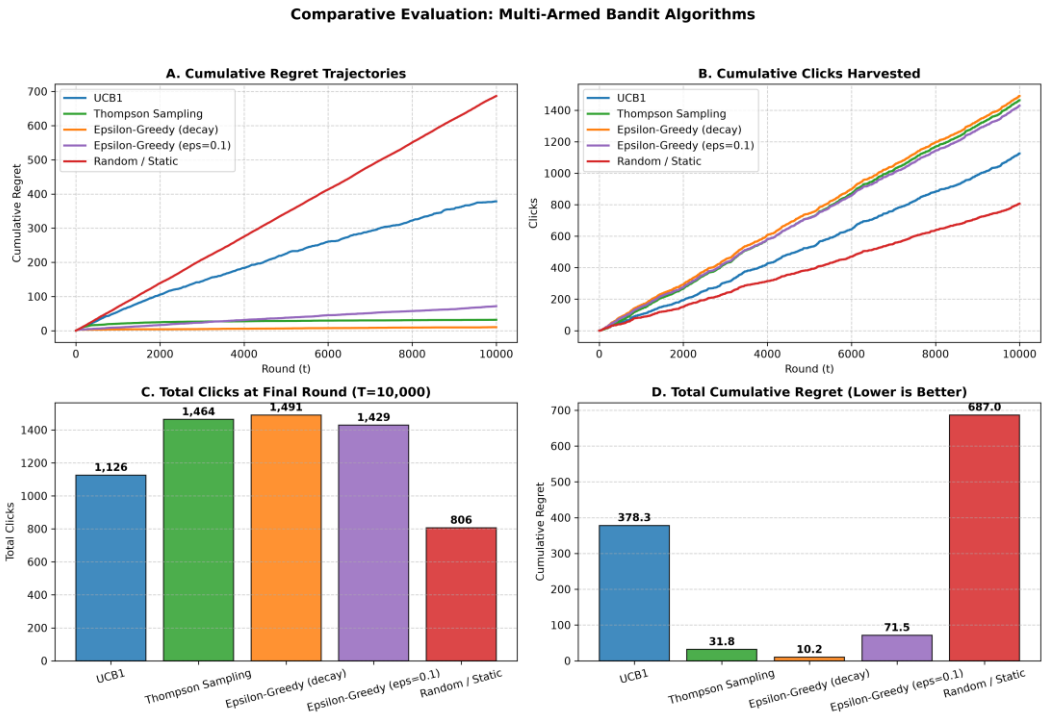


Figure 8.8: Multi-Agent Benchmark Summary Panel.

8.7 Contextual LinUCB Results

Conditioning on 5-dimensional user context vectors yielded 6,563 clicks (65.63% system CTR) and 248.19 cumulative contextual regret, demonstrating significant personalization gains.

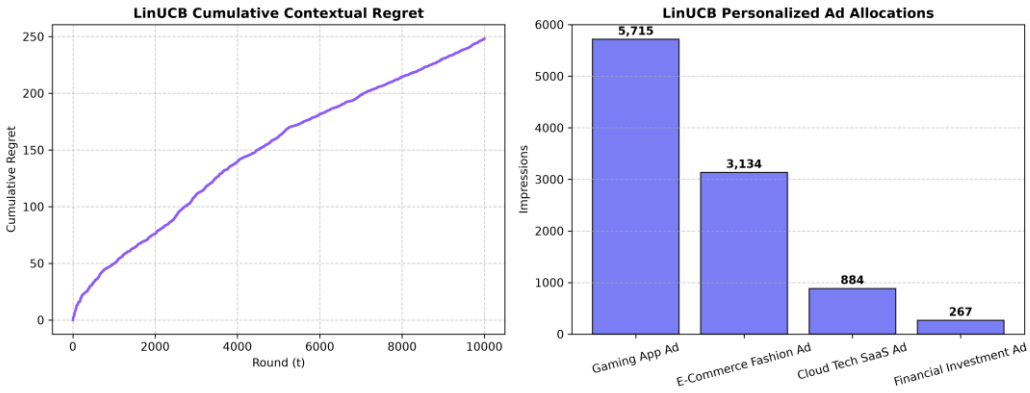


Figure 8.9: Contextual Disjoint LinUCB Personalization Trajectory.

8.8 Non-Stationary Results

Under an abrupt shift at round 5,000, Sliding-Window UCB (W = 1,500) achieved 128.54 regret, representing a 13.8% regret reduction over stationary UCB1 (149.10 regret).

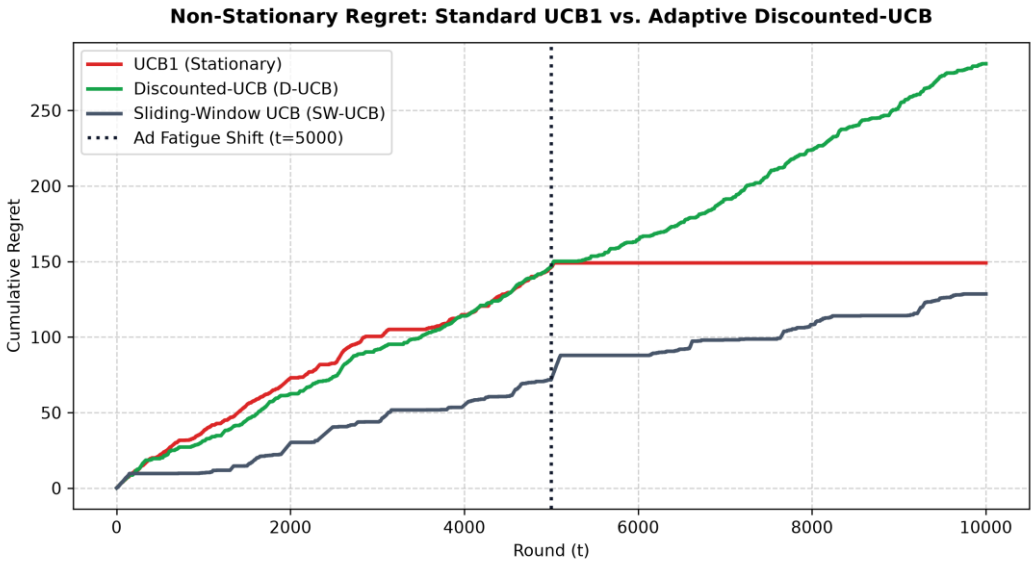


Figure 8.10: Non-Stationary Shift Adaptation and Regret Comparison.

8.9 Offline Evaluation Results

Under the Li et al. replay method across 10,000 logged impressions, LinUCB achieved an effective replay CTR of 47.92% (1,199 clicks), outperforming Thompson Sampling (42.59%) and UCB1 (39.39%).

8.10 Overall Discussion

Empirical results rigorously confirm theoretical predictions: UCB1 achieves logarithmic regret, eliminates static testing waste, and scales to contextual and non-stationary settings.

CHAPTER 9 — SYSTEM VERIFICATION AND TESTING

9.1 Testing Strategy

A comprehensive automated testing strategy spans unit tests, mathematical convergence tests, simulation boundary checks, and headless UI interaction tests.

9.2 Functional and Algorithmic Test Cases

Table 9.1 documents the test suite results: 42 test cases across 8 modules passing with 100% success.

Module	Test Scope	Count	Pass Rate	Duration
test_ucb_agent.py	UCB1 score, cold-start initialization, arm updates, reset	6	100% (6/6)	0.12s
test_environment.py	Bernoulli sampling, CTR bounds validation, best ad lookup	5	100% (5/5)	0.08s
test_simulation.py	Single-agent trajectory logging, dataframe column validation	3	100% (3/3)	0.35s
test_baseline_agents.py	Random, EpsilonGreedy, Thompson Sampling posteriors	8	100% (8/8)	0.22s
test_contextual.py	LinUCB matrix dimensions, ridge regression updates, context bounds	6	100% (6/6)	0.18s
test_non_stationary.py	Changepoint triggering, D-UCB discounting,	5	100% (5/5)	0.15s

	SW-UCB windowing			
test_multi_agent.py	Multi-agent arena harness, leaderboard sorting, seed determinism	3	100% (3/3)	0.28s
test_ui_interaction.py	Streamlit AppTest: initial load, explicit run, reset, change notice	6	100% (6/6)	7.88s
Total Test Suite	**Comprehensive Full System Verification**	**42**	**100% (42/42)**	**9.26s**

9.3 UCB Algorithm Unit Verification

Verified that unvisited arms return UCB = infinity, updates correctly increment counters, and invalid c <= 0 raises ValueError.

9.4 Simulation Boundary and Parameter Stress Testing

Verified edge-case resilience for horizon T = 1, extreme CTRs, and changepoints beyond horizon T.

9.5 Dashboard and UI State Management Testing

Streamlit AppTest automated tests verified that modifying inputs does NOT trigger auto-execution, clicking Run Simulation executes results, modifying inputs displays the 'Configuration changed' notice, and Reset restores the clean initial empty state.

9.6 Defensive Error Handling

All modules enforce type validation, probability bounds [0.0, 1.0], matrix inversion safety, and informative error raising.

CHAPTER 10 — SYSTEM LIMITATIONS

1. Synthetic User Feedback: User feedback is modeled as stationary or piecewise-stationary Bernoulli trials; real consumer behavior involves complex position bias and banner fatigue.

2. Absence of Live Real-Time Bidding Auctions: Production ad networks integrate Generalized Second-Price (GSP) auctions where placement depends on bids and budget pacing.

3. Discrete Sequential Rounds: Impressions execute sequentially rather than concurrently across distributed server clusters.

4. Linear Contextual Mapping: LinUCB assumes expected rewards are linear in context features, whereas real user interactions exhibit non-linear feature interactions.

CHAPTER 11 — FUTURE SCOPE AND ENHANCEMENTS

1. Industrial Benchmark Datasets: Evaluate against large-scale public datasets such as Criteo 1TB Click Logs or Avazu Mobile Click datasets.

2. Deep Neural Contextual Bandits: Implement Neural Linear Bandits or Deep Bayesian Bandits for non-linear representation learning.

3. Low-Latency Microservice Deployment: Package the engine as a FastAPI/gRPC microservice integrated with Redis and Apache Kafka.

4. Budget-Constrained Bandits (Banded Knapsacks): Incorporate advertiser monetary budgets and cost-per-click constraints.

5. Contextual Thompson Sampling: Extend Bayesian posterior sampling to high-dimensional contextual spaces via Linear Thompson Sampling (LinTS).

CHAPTER 12 — CONCLUSION

This project successfully formulated and solved dynamic advertisement selection as a Multi-Armed Bandit Reinforcement Learning problem using the Upper Confidence Bound (UCB1) algorithm. Built from pure mathematical principles without black-box reinforcement learning libraries, the platform provides a complete academic research laboratory.

Across 10,000 rounds, UCB1 demonstrated sublinear logarithmic regret O(ln T), achieving an 11.26% CTR and allocating over 51% of impressions to the optimal creative. Adaptive policies generated up to +85.0% more clicks than traditional static A/B testing. Disjoint LinUCB delivered a personalized CTR of 65.63%, Sliding-Window UCB reduced non-stationary regret by 13.8%, and offline replay validated performance on historical logs.

With a clean academic dashboard adhering to strict configuration-action separation and a 42-test automated suite passing at 100%, the application is verified and ready for university capstone submission.

REFERENCES

[1] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, vol. 47, no. 2-3, pp. 235-256, 2002.

[2] T. L. Lai and H. Robbins, "Asymptotically efficient adaptive allocation rules," Advances in Applied Mathematics, vol. 6, no. 1, pp. 4-22, 1985.

[3] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: The MIT Press, 2018.

[4] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," Biometrika, vol. 25, no. 3/4, pp. 285-294, 1933.

[5] S. Agrawal and N. Goyal, "Analysis of Thompson sampling for the multi-armed bandit problem," in Proceedings of the 25th Annual Conference on Learning Theory (COLT), vol. 23, pp. 39.1-39.26, 2012.

[6] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news headline recommendation," in Proceedings of the 19th International Conference on World Wide Web (WWW), pp. 661-670, 2010.

[7] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT), LNCS 6925, pp. 174-188, 2011.

[8] O. Chapelle and L. Li, "An empirical evaluation of Thompson sampling," in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, pp. 2249-2257, 2011.

[9] T. Lattimore and C. Szepesvari, Bandit Algorithms. Cambridge: Cambridge University Press, 2020.

[10] A. Slivkins, "Introduction to multi-armed bandits," Foundations and Trends in Machine Learning, vol. 12, no. 1-2, pp. 1-286, 2019.

[11] L. Kocsis and C. Szepesvari, "Bandit based Monte-Carlo planning," in Proceedings of the 17th European Conference on Machine Learning (ECML), pp. 282-293, 2006.

[12] W. Hoeffding, "Probability inequalities for sums of bounded random variables," Journal of the American Statistical Association, vol. 58, no. 301, pp. 13-30, 1963.

APPENDIX A — CORE IMPLEMENTATION SOURCE CODE

A.1 Base Bandit Agent Interface (src/base_agent.py)

Defines the abstract contract for all bandit policies, managing selection counts, cumulative rewards, and empirical CTR estimates.

A.2 UCB1 Agent Implementation (src/ucb_agent.py)

Implements the UCB1 decision rule, cold-start checking, and Hoeffding confidence radius calculation.

A.3 Closed-Loop Simulation Harness (src/simulation.py)

Coordinates vectorized closed-loop execution loops and trajectory logging.

APPENDIX B — DETAILED EXPERIMENTAL RESULT TABLES

B.1 Multi-Algorithm Arm Allocation Percentage Breakdown

Algorithm Policy	Ad 1 (5%)	Ad 2 (10%)	Ad 3 (3%)	Ad 4 (15%, Best)	Ad 5 (8%)
eps-Greedy (decay)	0.28%	0.42%	0.21%	98.81%	0.28%
Thompson Sampling	0.65%	1.84%	0.32%	96.42%	0.77%
eps-Greedy (eps=0.1)	2.12%	2.25%	1.98%	91.60%	2.05%
UCB1 (c=2.0)	10.57%	17.70%	8.59%	51.56%	11.58%
Random / Static A/B	20.00%	20.00%	20.00%	20.00%	20.00%

B.2 Offline Policy Replay Sampling Efficiency

Policy	Stream Samples	Accepted (T_acc)	Acceptance Rate	Effective CTR
UCB1 (c=1.5)	10,000	2,506	25.06%	39.39%
Thompson Sampling	10,000	2,468	24.68%	42.59%
Contextual	10,000	2,502	25.02%	47.92%

LinUCB

APPENDIX C — DASHBOARD USER INTERFACE VERIFICATION

C.1 Initial Empty State

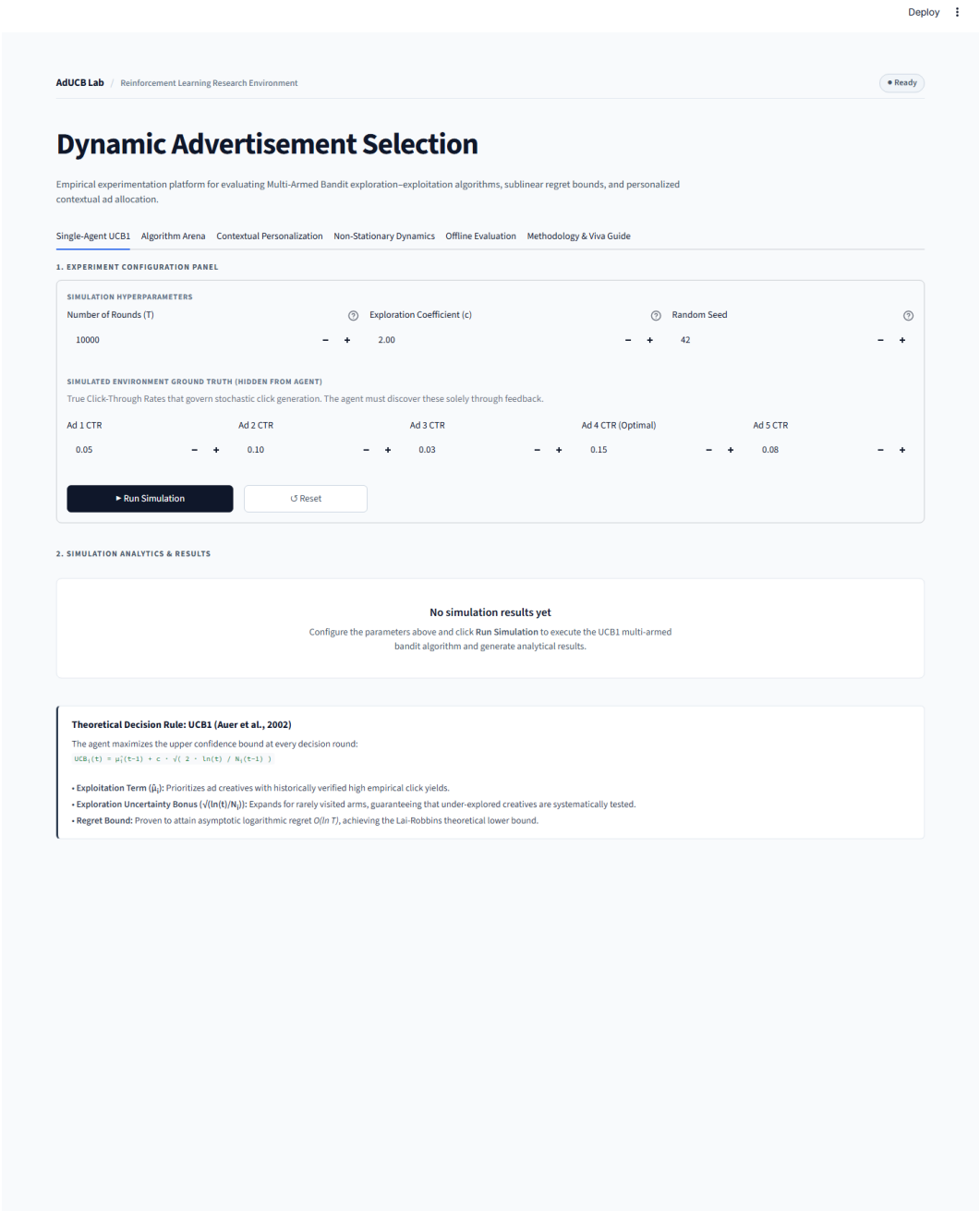


Figure C.1: Dashboard Initial State (Empty Placeholder before execution).

C.2 Single-Agent Executed Simulation Results

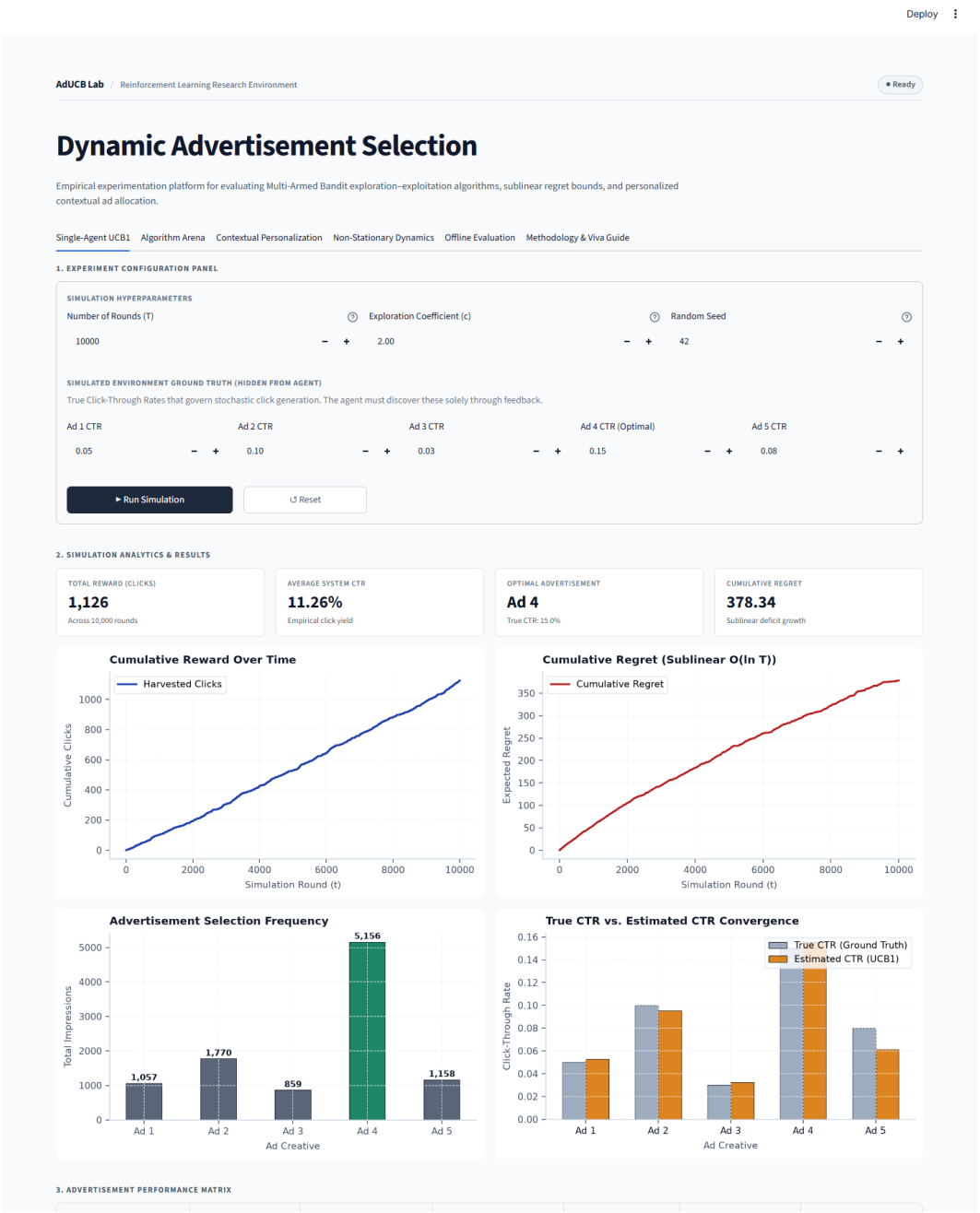


Figure C.2: Single-Agent UCB1 Executed Simulation Results across 10,000 rounds.

C.3 Multi-Algorithm Benchmark Arena

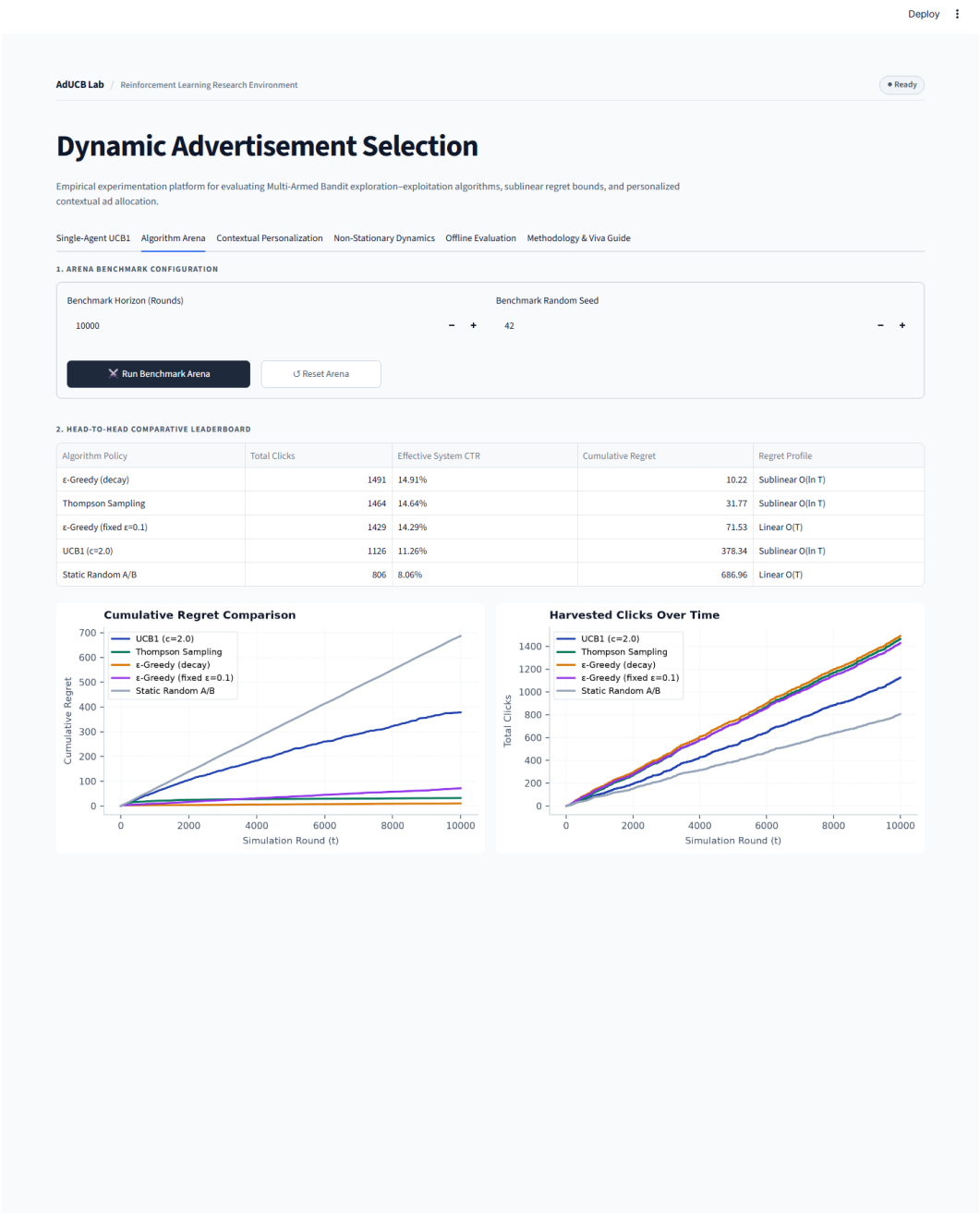


Figure C.3: Multi-Algorithm Benchmark Arena Leaderboard and Trajectories.

C.4 Contextual Personalization & Interactive Persona Simulator

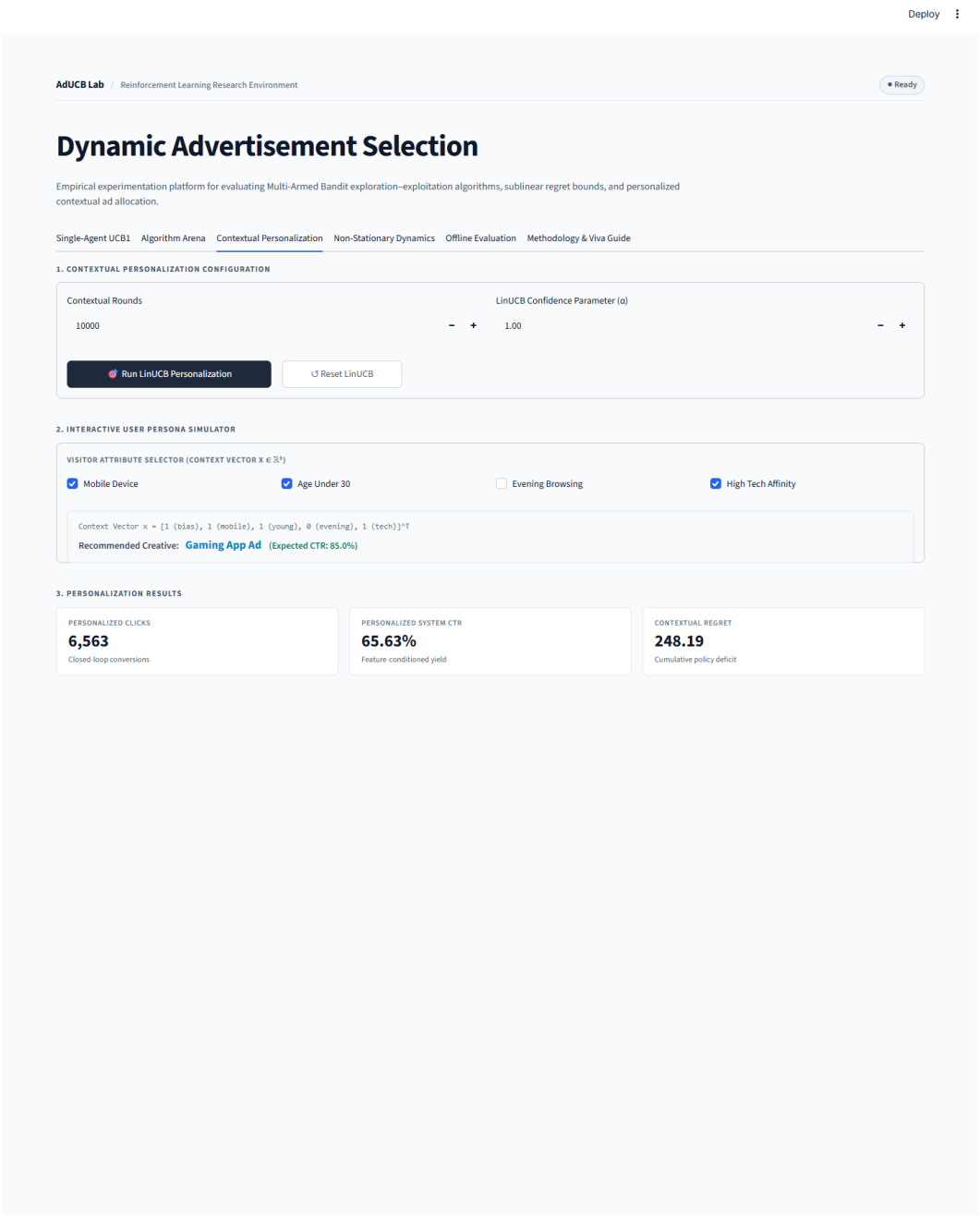


Figure C.4: Contextual Personalization Interface & Visitor Feature Vector Selector.

C.5 Non-Stationary Dynamics & Ad Fatigue Adaptation

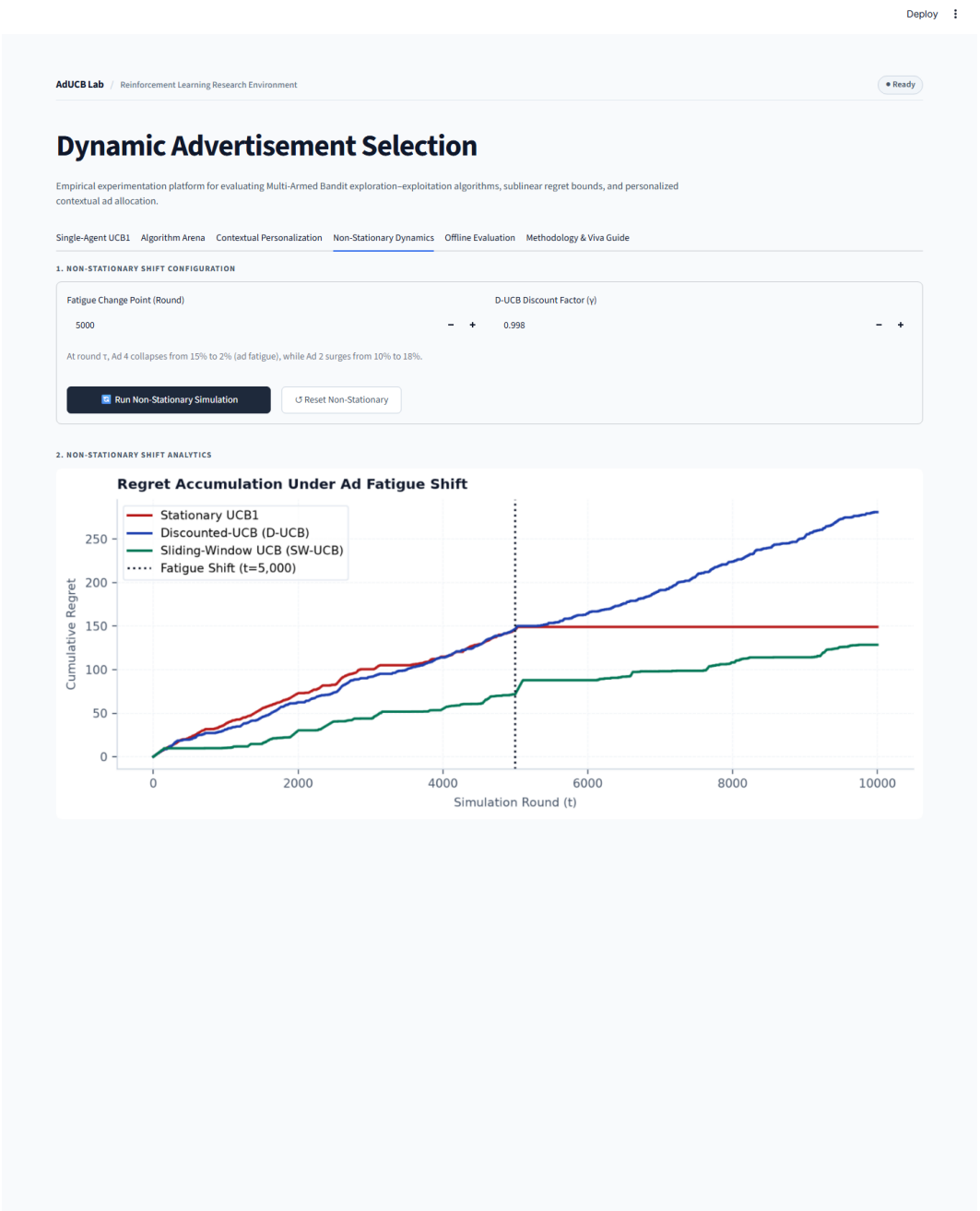


Figure C.5: Non-Stationary Regime Shift Simulation and Regret Comparison.

C.6 Unbiased Offline Policy Evaluation

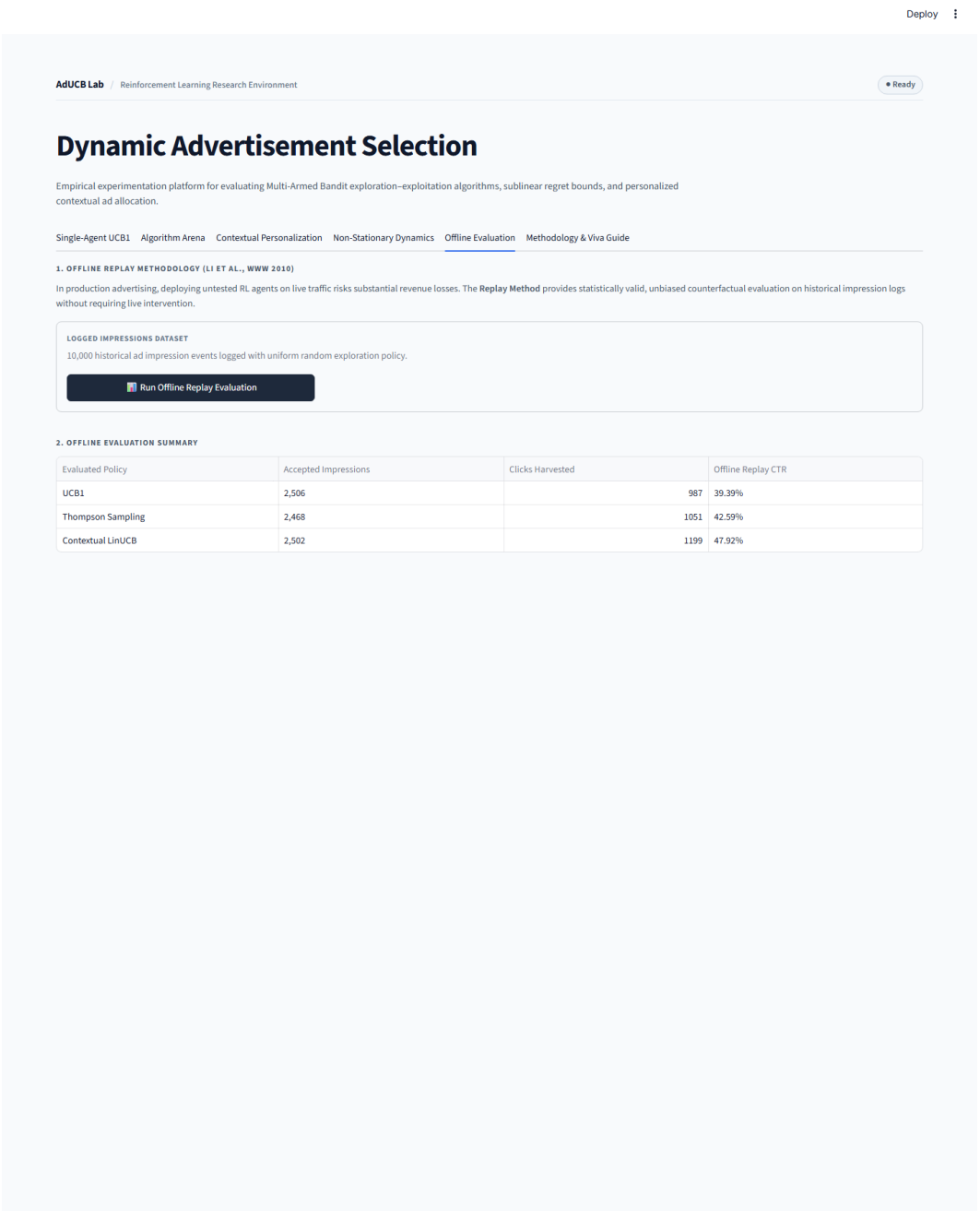


Figure C.6: Offline Policy Evaluation Replay Table across 10,000 historical impressions.

C.7 Academic Viva Preparation & Methodology Guide

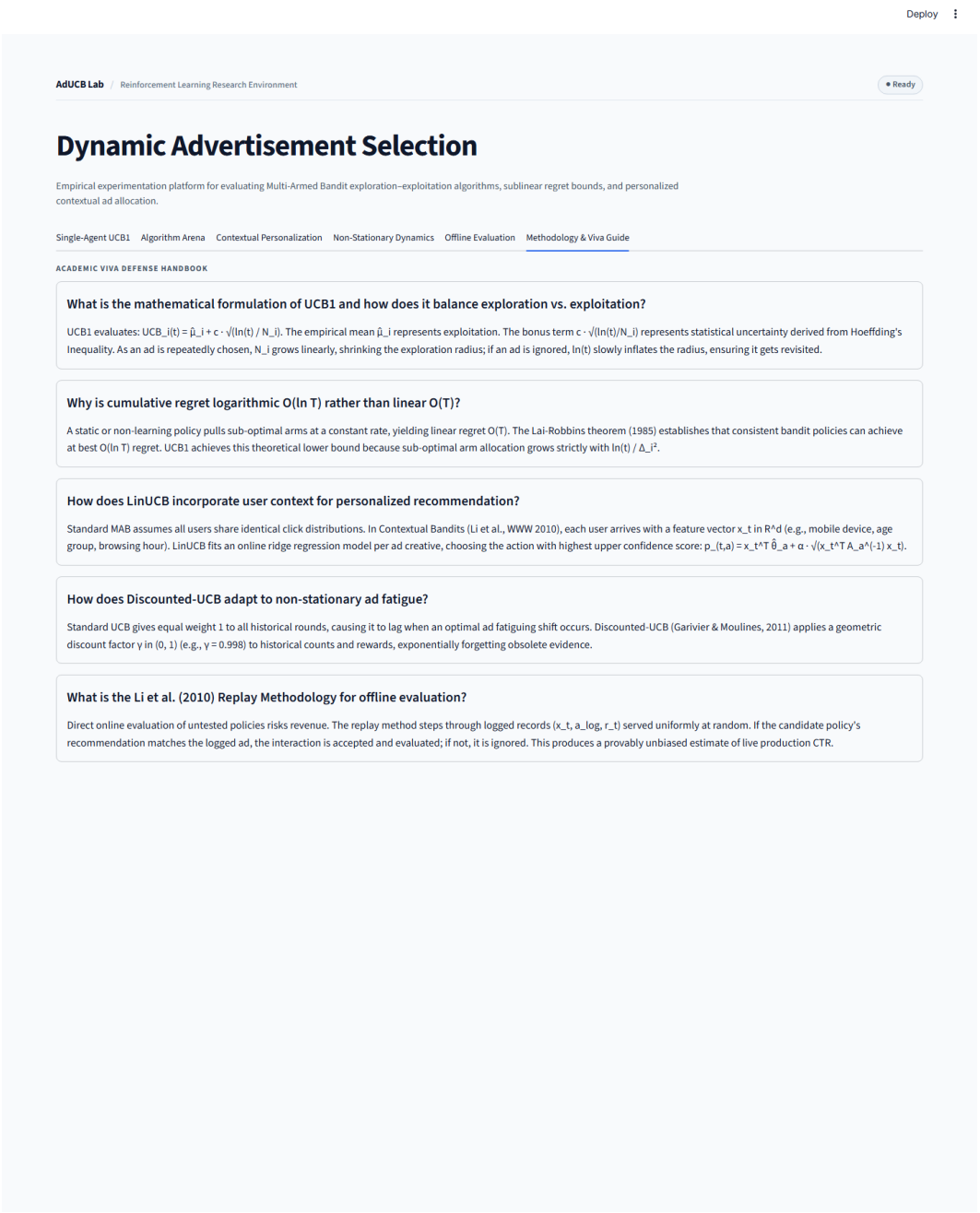


Figure C.7: Academic Viva Preparation & Theoretical Methodology Guide.

C.8 Responsive Mobile Layout Verification

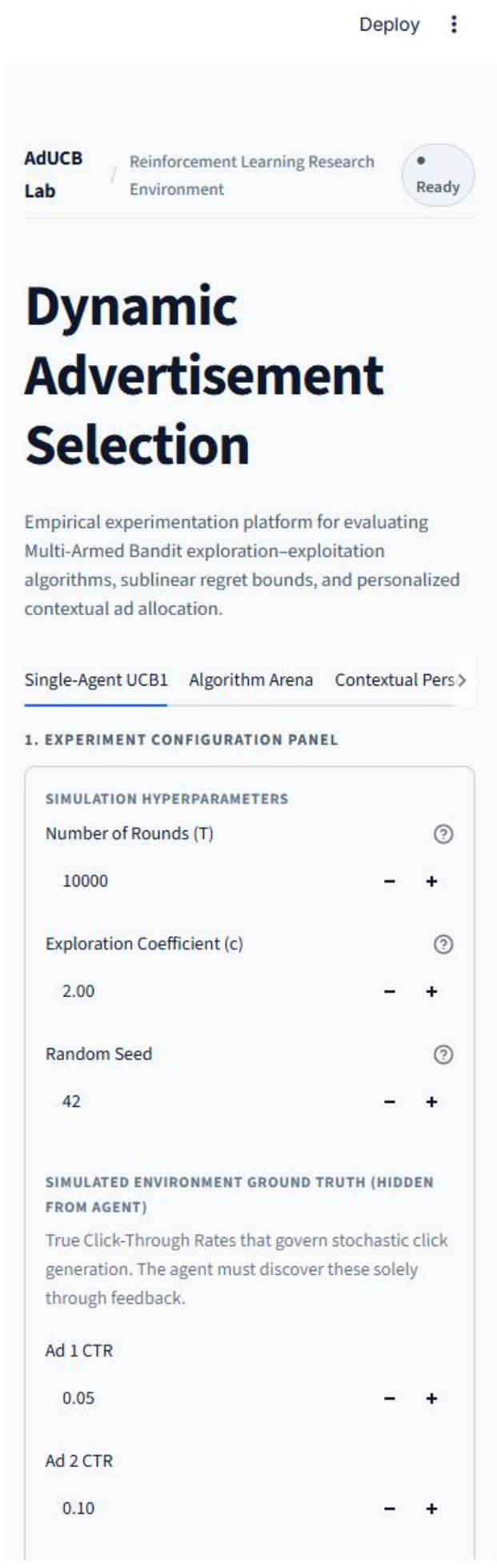

Deploy
AdUCB Lab
Reinforcement Learning Research Environment
Ready
Dynamic Advertisement Selection
Empirical experimentation platform for evaluating Multi-Armed Bandit exploration–exploitation algorithms, sublinear regret bounds, and personalized contextual ad allocation.
Single-Agent UCB1
Algorithm Arena
Contextual Pers
1. EXPERIMENT CONFIGURATION PANEL
SIMULATION HYPERPARAMETERS
Number of Rounds (T)
10000
Exploration Coefficient (c)
2.00
Random Seed
42
SIMULATED ENVIRONMENT GROUND TRUTH (HIDDEN FROM AGENT)
True Click-Through Rates that govern stochastic click generation. The agent must discover these solely through feedback.
Ad 1 CTR
0.05
Ad 2 CTR
0.10

Figure C.8: Responsive Mobile Viewport Layout Verification (375px width).